Context-Aware CPU Scheduling for Heterogeneous Workloads

BCSE303P – Operating Systems Lab

MINI-PROJECT COMPONENT REPORT

by

STUDENT NAME

(Register Number)

School of Computer Science and Engineering



OCTOBER 2026

DECLARATION

I hereby declare that the report entitled "**Context-Aware CPU Scheduling for Heterogeneous Workloads**" submitted by **STUDENT NAME**, for the BCSE303P Operating Systems Lab to VIT is a record of bonafide work carried out by me under the supervision of Dr. N. Narayanan Prasanth.

I further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for any other courses in this institute or any other institute or university.

Place : Vellore

Date : Signature of the Candidate

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty supervisor, Dr. N. Narayanan Prasanth, for their guidance, encouragement and valuable feedback throughout this mini-project. I thank the School of Computer Science and Engineering, Vellore Institute of Technology, for providing the opportunity and the resources to carry out this work as part of the BCSE303P Operating Systems Lab.

I also acknowledge the developers of the Linux kernel and of the open-source tools stress-ng, sysbench, fio and rt-tests, whose work made the experimental evaluation possible.

STUDENT NAME

ABSTRACT

General-purpose CPU schedulers treat batch, interactive and latency-sensitive programs alike unless an administrator configures them by hand. This project presents a context-aware scheduling layer that classifies running processes from their observable behaviour and gives each class a suitable share of the CPU. The idea was first studied in a discrete-time simulator with per-class time quanta, which reduced the 95th-percentile interactive response time by 34.8% with 79.2% classification accuracy. It was then implemented as a real Linux daemon, ctxschedd, which samples per-process scheduler counters from /proc every 100 ms (CPU time, run-queue wait, voluntary context switches and storage I/O), derives smoothed behaviour signals, classifies each process with rules whose thresholds are grounded in FreeBSD's ULE scheduler, the Linux EEVDF slice, kernel delay accounting and human-factors response-time limits, adds a confidence score, hysteresis and an anti-gaming guard, and moves it into a cgroup v2 group whose cpu.weight encodes the class. Evaluated on a Linux 6.6 (EEVDF) kernel under WSL2 with 10 repetitions per scenario, the daemon reduced the 95th-percentile wake-up delay of latency-sensitive tasks by 76.2% and interactive response time by 52.4%, at a cost of 19.5% in batch completion time. It followed behaviour changes within 0.50 s on average, recognised 100% of samples from batch jobs imitating latency-sensitive tasks (versus 0% without the guard), and classified unmodified benchmark tools correctly.

Keywords: CPU scheduling, EEVDF, CFS, cgroup v2, workload classification, Linux, latency.

CONTENTS

Page No.

Acknowledgement ..... **i**

Abstract ..... **ii**

Table of Contents ..... **iii**

List of Figures ..... **v**

List of Tables ..... **vi**

Abbreviations ..... **vii**

Symbols and Notations ..... **viii**

1 INTRODUCTION **1**

1.1 Objectives 1

1.2 Motivation 2

2 LITERATURE SURVEY **3**

2.1 Problem Statement 4

3 TECHNICAL SPECIFICATION **5**

3.1 Hardware and Software Environment 5

3.2 Kernel Interfaces Used 5

3.3 Requirements 5

4 DESIGN **7**

4.1 System Architecture 7

4.2 Control Group Hierarchy 7

4.3 Behaviour Signals 8

4.4 Core Classifier 9

4.5 Justification of the Classifier Thresholds 10

4.6 Simulator Design 14

5 PROPOSED SYSTEM **15**

5.1 Implementation Overview 15

5.2 Daemon Main Loop 15

5.3 Capturing Process IDs 16

5.4 Workloads with Known Ground Truth 16

5.5 Experimental Methodology 17

6 RESULTS AND DISCUSSION **18**

6.1 Simulator Results 18

6.2 Mixed Workload....19
6.3 Phase-Change Workload....20
6.4 Adversarial Workload and Ablation....21
6.5 Real Benchmark Programs....22
6.6 Classifier Accuracy Summary....23
6.7 Discussion....24
7 CONCLUSION....25
REFERENCES....26
APPENDIX....28

LIST OF FIGURES

Figure No.	Title	Page No.
4.1	Architecture of the context-aware scheduling daemon	7
4.2	cgroup v2 hierarchy used by the daemon	8
6.1	Simulator results: fixed quantum (baseline) versus context-aware quanta	18
6.2	Mixed workload: latency, interactive response and batch cost (error bars: 95% CI)…	19
6.3	Signals and daemon decision for a process that switches from batch to latency-sensitive behaviour	20
6.4	Adversarial workload: effect of the anti-gaming guard (error bars: 95% CI)	22
6.5	Confusion matrices of the classifier in the adversarial scenario, with and without the guard	22
6.6	Benchmark tools: improvement of the context-aware system over the baseline	23

LIST OF TABLES

Table No.	Title	Page No.
2.1	Comparison of approaches to workload-aware scheduling	4
3.1	Experimental platform	5
3.2	Linux interfaces read and written by the daemon	5
4.1	Mapping from class to scheduling treatment	8
4.2	Signals derived from kernel counters	8
4.3	Classification rules, checked in order	9
4.4	Classifier parameters: earlier value, adopted value and the evidence for it	11
4.5	Median measured signals per workload (adaptive runs) compared with the thresholds	12
5.1	Source files of the system	15
5.2	Synthetic workloads	16
5.3	Evaluation scenarios	17
6.1	Simulator: fixed quantum versus context-aware quanta	18
6.2	Mixed workload on the real kernel (mean ± 95% CI)	19
6.3	Phase-change workload (mean ± 95% CI)	20
6.4	Adversarial workload (mean ± 95% CI; Δ relative to baseline)	21
6.5	Benchmark tools (mean ± 95% CI)	23
6.6	Accuracy of applied decisions on the real kernel (after a 1 s warm-up per process)	23

LIST OF ABBREVIATIONS

CFS	Completely Fair Scheduler
CI	Confidence Interval
CPU	Central Processing Unit
CSV	Comma-Separated Values
EEVDF	Earliest Eligible Virtual Deadline First
EWMA	Exponentially Weighted Moving Average
I/O	Input/Output
IOPS	I/O Operations Per Second
MLFQ	Multi-Level Feedback Queue
PID	Process Identifier
p95 / p99	95th / 99th percentile
RR	Round Robin
TID	Thread Identifier
ULE	FreeBSD's default scheduler
VM	Virtual Machine
WSL2	Windows Subsystem for Linux, version 2

SYMBOLS AND NOTATIONS

Δt	sampling interval (100 ms)
α	EWMA smoothing factor (0.4)
$\hat{s}$	smoothed (EWMA) value of signal s
w	cgroup cpu.weight of a class
cpu_share	fraction of time a process ran on a CPU
blocked	fraction of wall time a process was asleep
sleep_frac	blocked / (blocked + cpu_share)
wake_s	wake-ups per second
burst_ms	CPU time per wake-up (ms)
io_mbps	storage throughput (MB/s)
io_wait	fraction of time waiting for block I/O
T	period of a periodic task, 1 / wake_s
m	margin of a classification rule
c	confidence of a decision, min(margins) capped at 1
N	hysteresis length (3 samples)
J	Jain's fairness index, $(\Sigma x_i)^2 / (n \cdot \Sigma x_i^2)$
$\bar{x} \pm t \cdot s/\sqrt{n}$	mean with 95% confidence interval over n repetitions

1. INTRODUCTION

A modern computer runs very different kinds of programs at the same time. A compiler or a video encoder wants as much CPU time as possible and does not mind waiting; a text editor or a web request handler sleeps most of the time but must respond quickly when it wakes up; an audio or control loop wakes up hundreds of times a second and must run almost immediately after every wake-up. General-purpose CPU schedulers such as Linux's Completely Fair Scheduler (CFS) and its successor EEVDF treat all of these the same way by default: every runnable task competes on an equal footing, with only a static, user-assigned priority (the *nice* value) to tell them apart.

This project builds a **context-aware scheduling layer** that observes how every managed process actually behaves at run time, classifies it as *batch*, *interactive* or *latency-sensitive*, and then gives each class different treatment from the kernel scheduler. The work was done in two stages. First, a discrete-time **simulator** (`scheduler.c`) was built to test the idea in a controlled setting, where the scheduler's time quantum could be changed per class. Second, the same pipeline was implemented as a **real Linux daemon** (`ctxschedd`) that reads live per-process scheduler statistics from `/proc`, classifies each process, and moves it into a cgroup v2 group whose `cpu.weight` encodes the class. The daemon was evaluated on a real Linux 6.6 kernel (running under WSL2 on Windows 11) with synthetic workloads of known type and with standard benchmark programs.

1.1. OBJECTIVES

- Design a lightweight pipeline — **signal collection → classification → mapping** — that identifies a process's workload type from its observable scheduling behaviour alone, without any hints from the application.
- Validate the idea in a simulator that compares a fixed-quantum baseline against per-class adaptive quanta.
- Implement the pipeline as a real user-space daemon that captures live process IDs and their kernel scheduler counters, and that changes how the kernel schedules them through cgroup v2 CPU weights.
- Make the classifier robust: a confidence score with a safe fallback, hysteresis against flapping, tracking of tasks that change behaviour, and a guard against tasks that try to game the classifier.

- Evaluate the system rigorously on a real kernel: repeated runs with 95% confidence intervals, four scenarios (mixed load, phase changes, adversarial tasks, real benchmark tools) and an ablation study.

1.2. MOTIVATION

The basic trade-off in CPU scheduling is between **responsiveness** and **throughput**. Short time slices let a task that has just woken up run sooner, which is what interactive and latency-sensitive programs need; long slices reduce context-switch overhead and cache disruption, which is what batch jobs need. A single global policy must pick one compromise for everybody. In practice, latency problems appear exactly when a machine is busy: a background build or data-processing job can delay an audio thread or a user-facing request by many milliseconds.

Earlier schedulers tried to guess interactivity inside the kernel (for example, the "sleep average" bonus of the Linux O(1) scheduler and the interactivity score of FreeBSD's ULE scheduler), but such heuristics were hard to tune and were removed from Linux when CFS arrived in 2.6.23. Today Linux offers the right *mechanism* — cgroup v2 CPU weights — but leaves the *policy* of who gets which weight to the administrator. This project explores whether that policy can be decided automatically, from behaviour, by a small daemon outside the kernel, and measures what it gains and what it costs.

2. LITERATURE SURVEY

Round-robin and the quantum trade-off. Classic textbooks describe round-robin scheduling with a fixed time quantum and show that the quantum controls the balance between response time and overhead: too long and the policy degenerates to first-come-first-served, too short and context switches dominate [1]. Our simulator uses exactly this model, with one quantum per class instead of one for all tasks.

Multi-level feedback queues (MLFQ). Corbató's Compatible Time-Sharing System introduced the idea of learning a job's nature from its past behaviour: jobs that use their whole quantum are moved to lower-priority queues with longer quanta, while jobs that give up the CPU early stay at high priority [2]. Arpaci-Dusseau and Arpaci-Dusseau show the weaknesses of this idea — starvation and *gaming* by a job that yields just before its quantum ends — and the fixes, priority boosting and accounting of total CPU use [3]. Our anti-gaming guard addresses the same attack.

Interactivity heuristics in production kernels. The Linux O(1) scheduler (2.6.0–2.6.22) gave tasks a dynamic priority bonus based on how long they slept, so that sleepers were treated as interactive [4]. FreeBSD's ULE scheduler computes an *interactivity score* from the ratio of a thread's voluntary sleep time to its run time [5]. Our `sleep_frac` signal is closely related to ULE's ratio; unlike both kernels, our classifier also considers wake-up frequency, burst length and I/O rate, reports a confidence, and runs in user space.

Proportional-share scheduling: CFS and EEVDF. CFS (Linux 2.6.23–6.5) gives each task CPU time in proportion to its weight by always running the task with the smallest weighted virtual runtime [6]. Stoica and Abdel-Wahab's *Earliest Eligible Virtual Deadline First* (EEVDF) algorithm [7] adds a virtual deadline to each request, so that tasks with short requests can be served sooner without receiving more than their share; it replaced CFS in Linux 6.6 [8]. With EEVDF, a heavier weight means both a larger CPU share and an earlier virtual deadline. Per-task custom slices for normal tasks only arrived in Linux 6.12 [9], so on the 6.6 kernel used here the class must be expressed through weights.

Control groups. cgroup v2 organises processes in a single hierarchy; the `cpu` controller's `cpu.weight` file (1–10000, default 100) sets a group's share relative to its siblings, and writing a PID to `cgroup.procs` moves a whole process into a group [10]. Per-task scheduler statistics are exported in `/proc/<pid>/schedstat` and `/proc/<pid>/status` [11], [12].

Behaviour-based classification and human factors. Research systems have identified interactive and multimedia processes from user-visible I/O [27], from I/O-channel activity [28], from the progress rate of real-rate applications [29], and with decision trees over process attributes [30]. Human-factors studies give the response-time limits that define "responsive": about 0.1 s to feel instantaneous and about 1 s to keep the user's flow of thought [23]–[25]. Section 4.5 uses these results, together with ULE's interactivity threshold and the Linux kernel's own definitions [18]–[22], to justify every threshold of our classifier.

Evaluation methodology. Jain's fairness index [13] is the standard measure of how evenly a resource is shared. Standard tools for scheduler evaluation include `cyclictest` for wake-up latency [14], `stress-ng` [15] and `sysbench` [16] for CPU load, and `fio` [17] for storage I/O.

Table 2.1: Comparison of approaches to workload-aware scheduling

Approach	How the type is learned	Where	Limitation
Fixed-quantum round robin [1]	not learned	kernel	one compromise for all tasks
MLFQ [2], [3]	quantum use history	kernel	starvation; gaming by yielding
O(1) sleep bonus [4]	average sleep time	kernel	hard to tune; removed in 2.6.23
FreeBSD ULE [5]	sleep/run ratio	kernel	single signal; kernel change needed
CFS / EEVDF + manual cgroups [6]–[10]	set by administrator	kernel + admin	no automatic policy
This project	4 measured signals + I/O rate, confidence, hysteresis, guard	user-space daemon	per-process (not per-thread); weights, not quanta

2.1. PROBLEM STATEMENT

Given a set of processes with different and possibly changing behaviour sharing a contended CPU, **automatically and continuously classify each process from its observable kernel scheduling statistics, and map each class to a scheduling treatment that improves the responsiveness of interactive and latency-sensitive processes, while (a) keeping the cost to batch throughput small and predictable, (b) adapting when a process changes behaviour, and (c) resisting processes that try to imitate a more favoured class.** The solution must run on an unmodified Linux kernel, without changes to the applications, and its benefit must be demonstrated with statistically sound measurements.

3. TECHNICAL SPECIFICATION

3.1. HARDWARE AND SOFTWARE ENVIRONMENT

Table 3.1: Experimental platform

Component	Specification
Host operating system	Windows 11 Home Single Language (build 26300)
Linux environment	WSL2 (Hyper-V virtual machine), Ubuntu 24.04.1 LTS
Kernel	6.6.87.2-microsoft-standard-WSL2 (EEVDF scheduler), cgroup v2 with cpu, io, memory, pids controllers, systemd
Logical CPUs visible to Linux	20
Compiler / runtime	GCC 13.3.0 (-O2 -Wall -Wextra), GNU Make, Python 3.12.3 (standard library only)
Benchmark tools	stress-ng 0.17.06, sysbench 1.0.20, fio 3.36, cyclictest (rt-tests) 2.50

3.2. KERNEL INTERFACES USED

Table 3.2: Linux interfaces read and written by the daemon

Interface	Access	Purpose
`/proc/<pid>/task/<tid>/schedstat`	read	time on CPU and time waiting in the run queue, per thread (ns)
`/proc/<pid>/task/<tid>/status`	read	`voluntary_ctxt_switches`: number of times the thread blocked
`/proc/<pid>/io`	read	`read_bytes`, `write_bytes`: storage I/O of the process
`/proc/<pid>/comm`	read	process name (used only to score accuracy)
`/sys/fs/cgroup/ctxsched/*/cgroup.procs`	read / write	discover managed PIDs; move a PID to its class group
`/sys/fs/cgroup/ctxsched/*/cpu.weight`	write	set the CPU weight of each class

3.3. REQUIREMENTS

Functional requirements

- Capture every process placed under management and sample its scheduler counters periodically (100 ms by default).

- Classify each process as batch, interactive or latency-sensitive, with a confidence value; fall back to the default class when unsure.
- Apply the class by moving the process to a cgroup with the matching CPU weight; never touch processes outside the managed hierarchy.
- Offer an observe-only mode for baseline measurements and an option to disable the anti-gaming guard for ablation.
- Log every sample for offline analysis; provide workloads with known ground truth, an experiment runner and an analysis tool.

Non-functional requirements

- No kernel modification and no change to the managed applications; runs on any kernel with cgroup v2 and `CONFIG_SCHED_INFO`.
- Not in the scheduling fast path: the daemon only adjusts weights, so it adds no cost to context switches.
- Results must be reproducible and reported with 95% confidence intervals over at least ten repetitions.

4. DESIGN

4.1. SYSTEM ARCHITECTURE

The system is a feedback loop around the kernel scheduler (Figure 4.1). Every 100 ms, the **signal collector** reads the cumulative counters of each managed process and turns the change since the previous sample into behaviour signals, which are smoothed with an exponentially weighted moving average (EWMA). The **heuristic classifier** maps the smoothed signals to a class and a confidence. A **hysteresis** stage accepts a new class only after three consecutive agreeing samples and replaces low-confidence decisions with the default class. Finally, the **weight mapper** moves the process to the cgroup of its class. The kernel then enforces the weights on its own; the new behaviour of the process is observed again on the next sample.

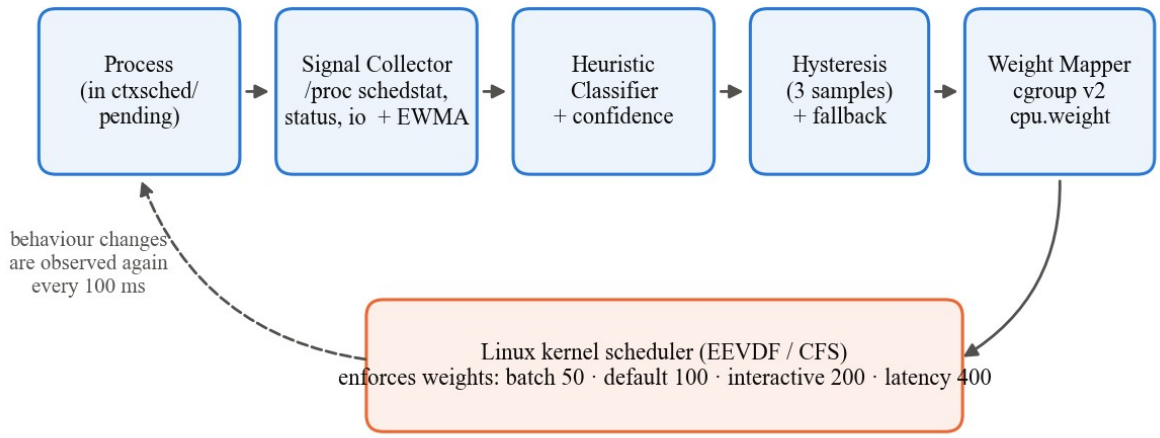


Figure 4.1: Architecture of the context-aware scheduling daemon

4.2. CONTROL GROUP HIERARCHY

All managed processes live under one cgroup, `/sys/fs/cgroup/ctxsched`, with the `cpu` controller enabled for its four children (Figure 4.2). New processes enter `pending`, which also serves as the default class. Because cgroup v2 shares CPU between *sibling* groups in proportion to their weights, a latency task in the weight-400 group competes against batch work in the weight-50 group with an 8:1 advantage, and under EEVDF also receives earlier virtual deadlines, so it is picked sooner after waking up.

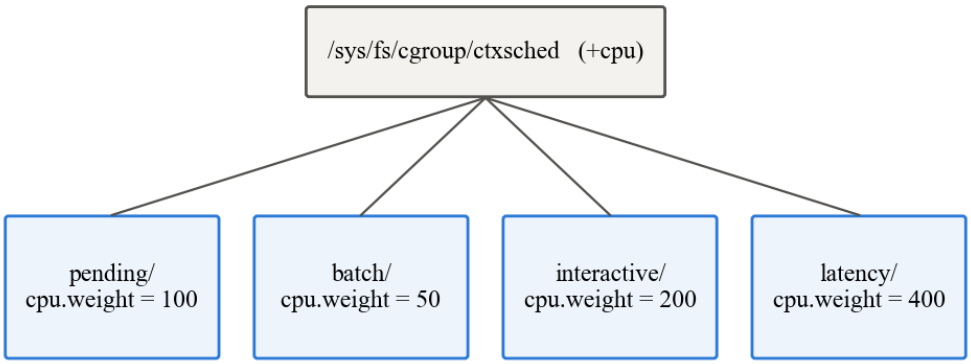


Figure 4.2: cgroup v2 hierarchy used by the daemon

Table 4.1: Mapping from class to scheduling treatment

Class	Simulator (quantum)	Real system (cgroup)	cpu.weight
batch	20 ms	ctxsched/batch	50
default / unsure	10 ms (baseline)	ctxsched/pending	100
interactive (incl. I/O-bound)	6 ms	ctxsched/interactive	200
latency-sensitive	2 ms	ctxsched/latency	400

Why weights instead of quanta. Kernels from 6.6 onwards use EEVDF, which has no per-task time quantum that user space can set (custom slices for normal tasks arrived only in 6.12). The simulator's short quanta for favoured classes are therefore translated into larger weights, which achieve the intended effect — earlier service after a wake-up — through virtual deadlines.

4.3. BEHAVIOUR SIGNALS

For each process the collector sums five cumulative counters over all its threads — CPU time and run-queue wait from `schedstat`, voluntary context switches from `status`, storage bytes from `io`, and block-I/O delay (field 42 of `stat`, from kernel delay accounting [21], [22]) — and computes, over the sampling interval Δt, the signals in Table 4.2. A process whose counters shrink because a thread exited is handled by clamping negative changes to zero.

Table 4.2: Signals derived from kernel counters

Signal	Definition	Meaning
cpu_share	ΔCPU / Δt	fraction of time running
blocked	$1 - \text{cpu_share} - \Delta\text{wait} / \Delta t$	fraction of time asleep (wall-clock)

Signal	Definition	Meaning
sleep_frac	blocked / (blocked + cpu_share)	of the time not spent waiting for a CPU, the fraction spent asleep
wake_s	Δvoluntary_ctxt_switches / Δt	wake-ups per second
burst_ms	ΔCPU / Δvoluntary_ctxt_switches	CPU time used per wake-up
io_mbps	Δ(read_bytes + write_bytes) / Δt	storage throughput (MB/s)
io_wait	Δdelayacct_blkio_ticks / CLK_TCK / Δt	fraction of time spent waiting for synchronous block I/O

Each signal is smoothed as $\hat{s} \leftarrow \alpha \cdot s + (1 - \alpha) \cdot \hat{s}$ with α = 0.4. The derived value **sleep_frac** is the key design decision of the classifier. On a busy CPU even a latency-sensitive task spends most of its wall time waiting in the run queue, so its *blocked* share falls; an early version of the anti-gaming guard used *blocked* and, under contention, misclassified real latency tasks as batch in 67% of samples. *sleep_frac* ignores run-queue waiting time, so it describes what the task *wants* to do and is the same at any load.

4.4. CORE CLASSIFIER

The core of the system is a small rule-based classifier, `classify()` in `daemon.c` (listed in Appendix B). It receives the smoothed signals of one process and applies the ordered rules of Table 4.3; the first rule that fires decides the class, and *batch* is the conservative default because the lowest weight is the safest mistake. A rule-based design was chosen over a learned model because each rule maps to a scheduling concept that can be checked against the literature (Section 4.5), it needs no training data, and its decisions can be explained sample by sample from the log.

Table 4.3: Classification rules, checked in order

#	Class	Condition	Reason
1	interactive (I/O-bound)	io_wait ≥ cpu_share and io_wait ≥ 0.01	the textbook definition of an I/O-bound process: more time waiting for I/O than computing [1]
2	latency	wake_s ≥ 40 and burst_ms ≤ base slice (3 ms) and **sleep_frac ≥ 0.625** (guard)	periodic work (period ≤ 25 ms) whose burst fits in one scheduler slice, from a task that genuinely sleeps
3	interactive	sleep_frac ≥ 0.625 and wake_s ≥ 1	mostly asleep (FreeBSD ULE's interactivity threshold), woken at least once a second
4	batch	otherwise	conservative default

If kernel delay accounting is unavailable, rule 1 falls back to *sleep_frac* ≥ 0.625 and *io_mbps* ≥ 1. On the test system the daemon enables delay accounting at start-up (`kernel.task_delayacct = 1`) and reads the scheduler's base slice from `/sys/kernel/debug/sched/base_slice_ns`.

4.4.1. Confidence and Fallback

Each condition has a *margin*: how far the signal lies beyond its threshold. Rate-type signals use a $\log_2$ scale (a margin of 1 means a factor of two beyond the threshold) and fractions use units of 0.2. A rule fires when all its margins are positive. The confidence *c* of a decision is the smallest margin involved — including how clearly the earlier rules failed — capped at 1. If $c < 0.25$ the process is kept in the default class instead of being trusted to a class.

4.4.2. Hysteresis

A process changes class only after the same candidate class wins N = 3 consecutive samples (about 300 ms). Together with the EWMA this prevents a process near a threshold from bouncing between groups, at the price of a short reaction delay when a process genuinely changes behaviour; Section 6.3 measures this delay.

4.4.3. Anti-Gaming Guard

A batch job can imitate a latency-sensitive task by inserting tiny sleeps between short bursts (for example 1.5 ms of computation and a 0.2 ms nap), producing hundreds of wake-ups per second with 1.5 ms bursts (about 600 per second when it runs alone, 74 per second measured under contention). Rule 2 alone would accept it and give it weight 400. The guard additionally requires *sleep_frac* ≥ 0.625: the imitator's measured sleep fraction is only 0.25 (Table 4.5), so it is classified as batch. To pass the guard a task must really sleep at least 62.5% of the time it is not waiting for the CPU, which gives up most of the CPU it was trying to obtain.

4.5. JUSTIFICATION OF THE CLASSIFIER THRESHOLDS

The first version of the classifier used thresholds chosen by hand. Each threshold was then checked against published scheduler designs, kernel definitions and human-factors research, and against measurements of our own workloads; four were changed as a result (Table 4.4). The aim was that every number in the classifier can be traced to a reason that is independent of this project.

Table 4.4: Classifier parameters: earlier value, adopted value and the evidence for it

Parameter	Earlier	Adopted	Evidence
Sleep threshold (rule 3, latency guard)	0.40 / 0.50	**0.625**	FreeBSD ULE interactivity threshold [5], [18]
Latency burst limit	2 ms	**base slice (3 ms)**	Linux EEVDF minimal preemption granularity [19]
Latency wake-up rate	50 /s	**40 /s**	audio/video periods [26]; 0.1 s perception limit [23]–[25]
Interactive wake-up rate	2 /s	**1 /s**	1 s flow-of-thought limit [23]–[25]
I/O-bound rule	sleep ≥ 0.4, ≥ 1 MB/s	**io_wait ≥ cpu_share**	definition of I/O-bound [1]; delay accounting [21], [22]
Confidence fallback	0.25	0.25	measured signal noise (CV 5–28%)
Hysteresis length N	3	3	measured raw error rate; false-switch probability
EWMA factor α	0.4	0.4	half-life ≈ 140 ms; reaction within the 1 s limit

4.5.1. Sleep Fraction Threshold: FreeBSD ULE

FreeBSD's ULE scheduler, in production since 2003, decides whether a thread is interactive from the ratio of its voluntary sleep time to its run time [5]. Its source code [18] states that this ratio "differs from the cpu usage because it does not account for time spent waiting on a run-queue" — exactly the property of our *sleep_frac*. ULE scores a thread as $score = 50 \cdot run / sleep$ when it sleeps more than it runs, and treats it as interactive when the score is below the threshold `SCHED_INTERACT_THRESH` = 30. Solving 50·run/sleep < 30 gives run/sleep < 0.6, which is

$$sleep_frac = sleep / (sleep + run) > 1 / 1.6 = 0.625.$$

We adopt 0.625 both for the interactive rule and for the latency guard, replacing the earlier 0.40 and 0.50. The stricter guard makes imitation more expensive: an adversary now has to sleep 62.5% of its non-waiting time instead of 50%.

4.5.2. Latency Burst Limit: the Kernel's Own Slice

In Linux 6.6 the EEVDF scheduler gives each request a slice of `sysctl_sched_base_slice`, documented in `fair.c` as the "minimal preemption granularity for CPU-bound tasks", with a default of 0.75 ms × (1 + ilog_2(CPUs)), the CPU count being capped at 8 [19]. On the test machine the kernel reports 3,000,000 ns. A task whose CPU need per wake-up fits within one slice completes each burst before its virtual deadline and is never preempted mid-burst; this is

the class of task for which an earlier deadline (a higher weight) translates directly into lower latency. The daemon therefore reads the slice from the kernel instead of using a fixed 2 ms, and computes it with the kernel's formula when debugfs is not mounted.

4.5.3. Wake-up Rate Thresholds: Human Perception and Media Periods

Human-factors research gives two classic limits for response time [23]–[25]: about **0.1 s** for a system to feel instantaneous, and about **1 s** for the user's flow of thought to stay uninterrupted. Latency-sensitive software runs periodic work far below the first limit: PipeWire, the default Linux audio server, processes audio in quanta of 1024 frames at 48 kHz by default [26], i.e. every 21.3 ms (≈ 47 wake-ups/s), while video and display loops run at 50–60 Hz (16.7–20 ms). The earlier threshold of 50 wake-ups/s would have excluded the default audio configuration; the adopted **40 wake-ups/s** (period ≤ 25 ms, a quarter of the 0.1 s limit) includes it and the frame loops. For the interactive class, the 1 s limit gives the lower bound of **1 wake-up/s**: a task that is woken less than once per second has no responsiveness requirement at human time scales that a scheduling weight could improve.

4.5.4. I/O-Bound Rule: Definition and Delay Accounting

Operating-systems textbooks define an I/O-bound process as one that spends more of its time doing I/O than doing computation [1]. The Linux kernel measures the relevant quantity directly: with delay accounting enabled, it records for every task the time spent "waiting for synchronous block I/O to complete" [21], exposed per thread as field 42 of `/proc/<pid>/task/<tid>/stat` [22]. Rule 1 is therefore the definition itself — *io_wait* ≥ *cpu_share* — with a 1% floor so that tasks that neither compute nor wait (idle tasks) are not called I/O-bound. This replaces the earlier rule based on 1 MB/s of storage traffic, which had no basis beyond our own workload. Table 4.5 shows the measured signals of every workload in the experiments: fio waits for the disk 0.699 of the time while computing only 0.042 (as fractions of wall time), whereas every CPU-bound and latency-sensitive program has an I/O wait of zero.

Table 4.5: Median measured signals per workload (adaptive runs) compared with the thresholds

Workload	cpu_share	sleep_frac	wake_s	burst_ms	io_wait
batch (synthetic)	0.115	0.243	0.0	11.483	0.000
stress-ng	0.286	0.023	0.0	28.564	0.000
sysbench	0.286	0.018	0.0	28.614	0.000
adversary	0.111	0.248	73.9	1.504	0.000
interactive (synthetic)	0.079	0.910	8.3	7.795	0.000

Workload	cpu_share	sleep_frac	wake_s	burst_ms	io_wait
fio	0.042	0.956	5315.5	0.008	0.699
latency (synthetic)	0.130	0.857	199.3	0.657	0.000
cyclictest	0.009	0.991	1099.2	0.008	0.000
threshold	—	0.625	40 / 1	3.0	≥ cpu_share

Every workload sits clearly on its own side of each threshold — typically by a factor of two or more — which is why the classifier's accuracy is insensitive to the exact values. The adversary is the instructive exception: it passes the wake-up and burst tests of the latency rule, and only the sleep fraction (0.25 against 0.625) exposes it. fio is the closest call for rule 1: its median I/O wait is many times its CPU share, but in some baseline samples — when fio is starved of CPU — the two briefly become similar (≈ 0.04 each). Those samples carry low confidence and are absorbed by the fallback and hysteresis; after hysteresis fio was classified correctly in 98% of baseline samples and 100% of adaptive samples.

4.5.5. Data-Fitted Parameters: Confidence, Hysteresis and Smoothing

Three parameters control noise rather than class boundaries, so they were checked against the measured noise of the signals over 107,337 samples of correctly-behaving processes.

- **Confidence fallback (0.25).** The median coefficient of variation of the smoothed wake-up rate is 5% (90th percentile 28%), and of the burst length 4%. A margin of 0.25 corresponds to ±19% on a rate threshold (2^0.25 = 1.19) and ±0.05 on the sleep fraction. This band lies between the typical noise (median CV ≈ 5%) and the noise of the most variable workloads (90th percentile ≈ 28%), so a decision that close to a threshold could be reversed by ordinary measurement noise; such decisions are treated as uncertain and given the default weight.
- **Hysteresis (N = 3).** Away from a class boundary, the highest raw per-sample error rate of any correctly-behaving workload was 0.089% (for workloads with no observed error, the 95% upper bound 3/n of the "rule of three" was used); fio, whose errors cluster at the I/O boundary described above, is handled by the confidence fallback instead. A false class switch requires N wrong samples in a row, with probability p^N: about 8.0×10^{-7} for N = 2 (one false switch every 1.5 days per process at 10 samples/s) and 7.1×10^{-10} for N = 3 (one every 4 years). N = 3 is the smallest value that makes spurious switching negligible.

- **Smoothing (α = 0.4).** The half-life of the EWMA is ln 0.5 / ln 0.6 ≈ 1.4 samples ≈ 140 ms. Combined with the hysteresis this gives the reaction time measured in Section 6.3 (about 0.5–0.6 s), which stays below the 1 s flow-of-thought limit [23].

4.5.6. Class Weights in Kernel Terms

The kernel's `sched_prio_to_weight` table changes a task's weight by a factor of 1.25 per nice level ("a gentle 10% change for every nice level") [20]. The class weights therefore correspond to familiar priorities: latency (×4) ≈ nice −6, interactive (×2) ≈ nice −3, default = nice 0, and batch (×0.5) ≈ nice +3. These are moderate adjustments within the normal range that an administrator could apply by hand; the daemon only automates the choice.

4.5.7. Related Work on Behaviour-Based Classification

Other research has classified processes from their behaviour with different signals. Etsion, Tsafrir and Feitelson identify interactive and multimedia processes by the user-visible I/O they produce (keyboard, mouse and screen output) and show that CPU-usage-based heuristics fail for CPU-intensive multimedia [27]; RSIO by Zheng and Nieh detects user interactions from I/O channel activity and boosts the processes involved, including indirectly involved ones [28]; Steere et al. infer the required CPU proportion and period of real-rate applications with a feedback controller [29]; and Negi and Kumar train decision trees on process attributes to predict CPU bursts in Linux [30]. Our classifier differs in using only standard per-task kernel counters, which makes it portable to any modern Linux kernel without kernel changes or application hooks; the cited works confirm that sleep/run behaviour, I/O activity and periodicity are the established signals for this task.

4.6. SIMULATOR DESIGN

The simulator (`scheduler.c`) models 24 tasks (6 batch, 10 interactive, 8 latency-sensitive) with randomly drawn arrival times, work and behaviour parameters, dispatched by a round-robin loop with a 0.05 ms context-switch cost. The baseline gives every task a 10 ms quantum; the adaptive scheduler runs collect → classify → quantum on every dispatch. In the simulator the signals are the task's own parameters plus ±5% noise, so it is a controlled design study; the real daemon replaces this with measured signals.

5. PROPOSED SYSTEM

5.1. IMPLEMENTATION OVERVIEW

Table 5.1: Source files of the system

File	Language	Role
scheduler.c	C	discrete-time simulator: task model, classifier, quantum mapper, metrics
linux/daemon.c	C	ctxschedd: signal collector, classifier, hysteresis, cgroup writer, logging
linux/workload.c	C	test programs with known behaviour and their own latency measurement
linux/setup.sh	Bash	checks kernel support, creates the cgroups, installs tools, builds
linux/run_experiment.sh	Bash	runs scenarios in baseline / adaptive / noguard modes, repeatedly
linux/analyze.py	Python	metrics, 95% confidence intervals, accuracy, confusion matrix, reaction time

5.2. DAEMON MAIN LOOP

The daemon is a single C program without external dependencies. Its main loop, executed on an absolute 100 ms timer so that the sampling cadence does not drift, is:

```
every 100 ms:
  scan_cgroups()                # read cgroup.procs of pending, batch, interactive,
latency
  for each managed PID:
    read_comm()                 # name, for scoring only
    read_counters()             # sum schedstat + status over threads, read /proc/<pid>/io
    s = signals_from_deltas()   # cpu_share, blocked, wake_s, burst_ms, io_mbps
    ewma_update(s)
    raw, conf = classify(ewma)
    want = raw if conf >= 0.25 else DEFAULT
    streak = streak + 1 if want == candidate else 1;  candidate = want
    if candidate != current and streak >= 3:
      current = candidate
      move_to_class(pid, current) # write PID to ctxsched/<class>/cgroup.procs
    log one CSV row
```

Command-line options select the mode: no option runs the adaptive policy; `--observe` classifies and logs without moving anything (the **baseline**, in which every task keeps weight 100); `--no-guard` disables the anti-gaming guard (ablation); `--interval`, `--duration` and `--log` control sampling, run time and the per-sample log.

5.3. CAPTURING PROCESS IDS

Management is opt-in, so the daemon never interferes with system services. The life of a process ID under the daemon is:

1. **Entry.** The process's PID is written to `ctxsched/pending/cgroup.procs`. The experiment runner does this in a small wrapper that writes its own PID (`echo $$`) and then `exec`s the real program, so the program keeps that PID. Any existing process can be added the same way, and its children are born in the same group automatically.
2. **Discovery.** On every tick the daemon reads `cgroup.procs` of all four class groups; a PID not seen before gets an entry in the process table, with its current group as its current class.
3. **First sample.** Only the counters are recorded, because signals need two samples.
4. **Every later tick.** Signals are computed, smoothed and classified, hysteresis is applied, and the PID is moved if its class changed.
5. **Exit.** When a PID no longer appears in any managed group (it exited or was moved out), its entry is freed.

5.4. WORKLOADS WITH KNOWN GROUND TRUTH

To measure accuracy and benefit, `workload.c` provides programs whose type is known. Each one burns *CPU time* rather than wall time (so preemption does not shrink its work), names itself with `prctl(PR_SET_NAME)` — the name is the ground truth used for scoring only — and measures its own performance (Table 5.2).

Table 5.2: Synthetic workloads

Workload	Behaviour	Measured
wl_batch	computes continuously, never sleeps	completion time, or CPU obtained in a fixed time
wl_interactive	sleeps 50–150 ms, then a 5–15 ms burst	response time: intended wake-up → burst finished
wl_latency	absolute 5 ms timer, 0.3–1 ms burst	wake-up latency: actual wake-up − timer deadline
ph_<class>	behaves as one class, then switches to another half-way and renames itself	latency of the interactive/latency phase; classifier reaction time
adv_batch	batch work cut into 1.5 ms bursts with 0.2 ms naps (imitates latency)	completion time; whether the classifier is fooled

5.5. EXPERIMENTAL METHODOLOGY

Two measures make the comparison fair. First, **contention is forced**: all tasks are pinned to CPU core 0 with `taskset`, and the daemon runs on core 1. On an idle 20-CPU machine every task would get its own core and weights would change nothing. Second, the **modes alternate within each repetition** (baseline, adaptive and, where used, noguard), so slow changes in the host affect every mode equally. Every scenario is repeated 10 times with 20-second runs, and every metric is reported as the mean over repetitions with a 95% confidence interval $\bar{x} \pm t_{0.975,n-1} \cdot s/\sqrt{n}$.

Table 5.3: Evaluation scenarios

Scenario	Processes on core 0	Question answered
mixed	4 batch, 3 interactive, 3 latency	Do interactive and latency tasks improve, and what does batch pay?
phase	2 batch → latency, 2 interactive → batch, 3 background batch, 1 latency	Does the classifier follow behaviour changes, and how fast?
adversarial	2 adversaries, 2 batch, 2 interactive, 3 latency; modes baseline / adaptive / noguard	Can a batch job game the classifier, and does the guard stop it?
tools	stress-ng --cpu 2, sysbench cpu, fio (4 KiB random synchronous direct writes), cyclictest (1 ms timer), 1 interactive	Does the approach hold for real, unmodified programs?

6. RESULTS AND DISCUSSION

6.1. SIMULATOR RESULTS

Table 6.1 and Figure 6.1 show the simulator's comparison of the fixed 10 ms quantum against per-class quanta. The 95th-percentile response time of interactive and latency tasks falls by 34.8% while batch completion and makespan are essentially unchanged. The classifier, operating on noisy versions of each task's true parameters, reaches 79.2% accuracy; its main confusion is between interactive and latency tasks, whose parameter ranges overlap. The fairness metric (standard deviation of per-class CPU share) worsens by 50.5%, because the short quanta let latency tasks finish very quickly relative to their small amount of work.

Table 6.1: Simulator: fixed quantum versus context-aware quanta

Metric	Baseline	Context-aware	Change
Interactive p95 response (ms)	271.86	177.20	−34.8%
Interactive mean response (ms)	153.62	151.00	−1.7%
Batch mean completion (ms)	8148.85	8138.13	−0.1%
Fairness: CPU-share std. dev.	0.05	0.07	+50.5%
Makespan (ms)	9261.55	9246.45	−0.2%
Classifier accuracy	—	79.2%	—

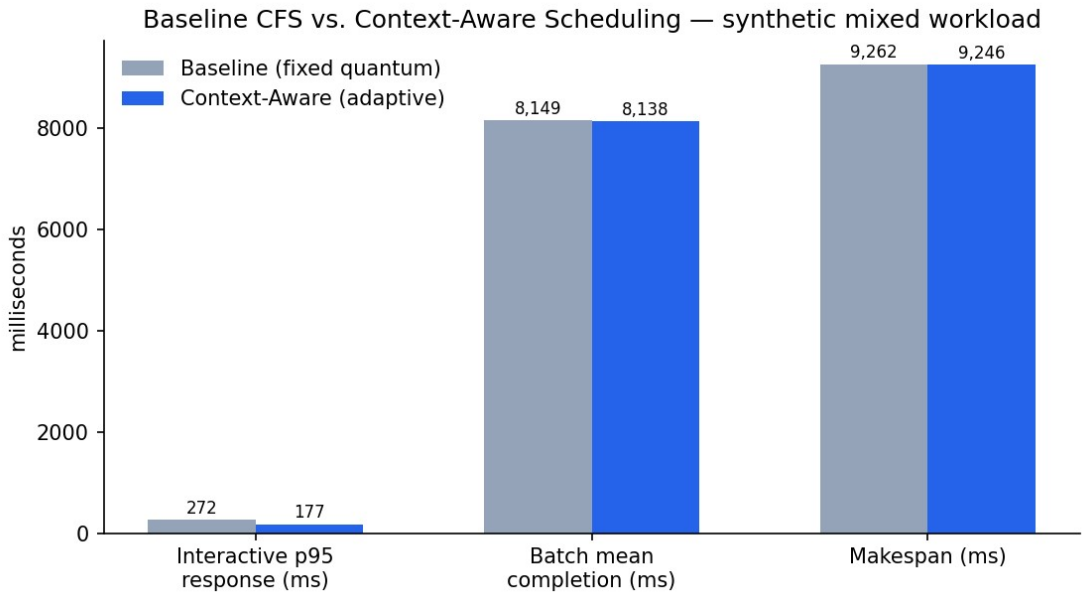


Figure 6.1: Simulator results: fixed quantum (baseline) versus context-aware quanta

Because the simulator's signals are partly given rather than measured, these numbers are a design study. The following sections present the evidence from the real kernel.

6.2. MIXED WORKLOAD

Table 6.2 and Figure 6.2 give the main result on the real kernel (10 repetitions, mean ± 95% CI). With the daemon active, the 95th-percentile wake-up delay of the latency tasks falls from 11.2 ms to 2.7 ms (−76.2%), and the p95 response time of the interactive tasks from 113.2 ms to 53.9 ms (−52.4%). The confidence intervals do not overlap, so the improvements are clearly real. The cost is paid by the batch tasks, whose mean completion time grows by 19.5%; within the batch class the CPU remains perfectly evenly shared (Jain index 1.000). The classifier was correct in 100.0% of 20,611 samples.

Table 6.2: Mixed workload on the real kernel (mean ± 95% CI)

Metric	Baseline	Context-aware	Change
Latency wake-up p95 (ms)	11.25 ± 0.42	2.68 ± 0.31	−76.2%
Latency wake-up p99 (ms)	13.64 ± 0.38	4.68 ± 0.38	−65.7%
Latency wake-up max (ms)	18.55 ± 0.67	14.70 ± 0.75	−20.7%
Interactive response p95 (ms)	113.21 ± 1.22	53.87 ± 1.94	−52.4%
Interactive response mean (ms)	75.88 ± 0.78	28.09 ± 0.92	−63.0%
Batch mean completion (s)	20.41 ± 0.06	24.38 ± 0.13	+19.5%
Batch fairness (Jain index)	1.000 ± 0.000	1.000 ± 0.000	−0.0%

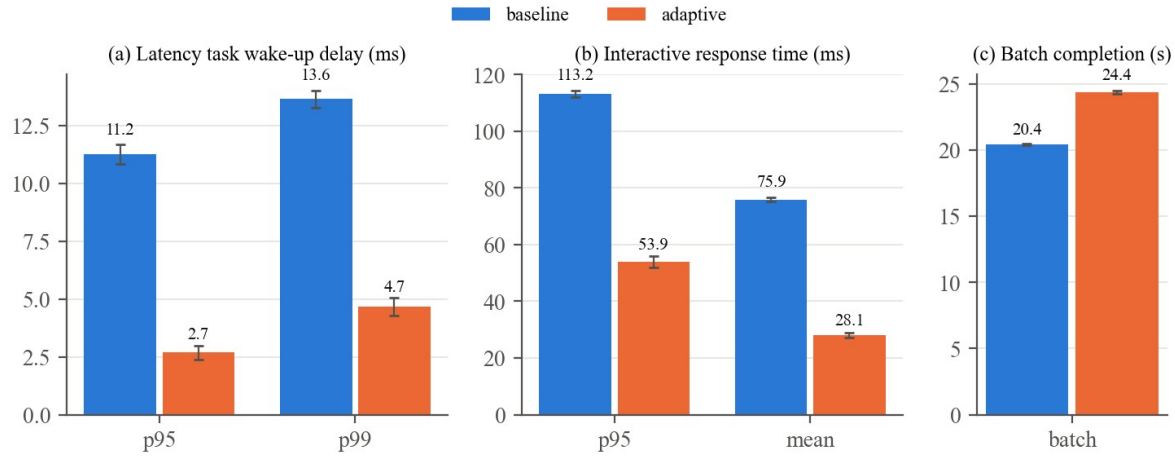


Figure 6.2: Mixed workload: latency, interactive response and batch cost (error bars: 95% CI)

The worst-case (maximum) wake-up delay improves much less (−20.7%) than the percentiles. A cgroup weight changes the *share* and the *deadline* a task receives, not a hard guarantee, so occasional long delays caused by the hypervisor and by the batch tasks' slices remain.

6.3. PHASE-CHANGE WORKLOAD

In this scenario four processes change behaviour half-way through each run. Figure 6.3 shows one batch → latency process from the daemon's log: while it computes, its wake-up rate is zero and its sleep fraction is low, and the daemon keeps it in the batch class; when it switches, the wake-up rate jumps to 200 per second and the sleep fraction to about 0.85, and once the moving averages have crossed the thresholds (40 wake-ups/s, sleep fraction 0.625) and three samples agree, the decision moves to the latency class. Over all 40 switches the daemon needed on average 0.50 s (at most 0.70 s) to adopt the new class, which matches the EWMA settling time plus three hysteresis samples.

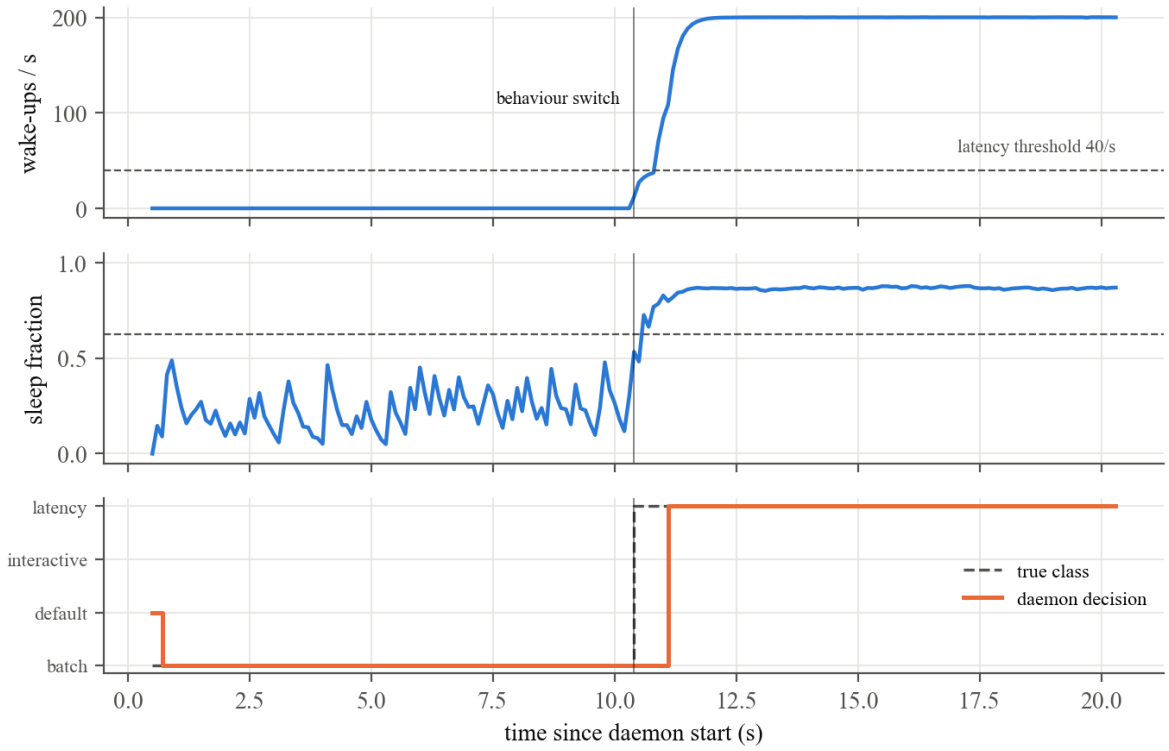


Figure 6.3: Signals and daemon decision for a process that switches from batch to latency-sensitive behaviour

Table 6.3: Phase-change workload (mean ± 95% CI)

Metric	Baseline	Context-aware	Change
Steady latency task: wake-up p95 (ms)	10.15 ± 0.41	1.63 ± 0.40	−83.9%
Batch → latency task: wake-up p95 (ms)	10.13 ± 0.70	1.37 ± 0.37	−86.5%
Interactive → batch task: response p95 (ms)	101.94 ± 1.62	29.81 ± 1.91	−70.8%

Metric	Baseline	Context-aware	Change
Background batch CPU obtained (ms)	3073 ± 20	2586 ± 8	−15.9%

Processes that become latency-sensitive benefit just as much as processes that always were (wake-up p95 −86.5%), and processes that start interactive are served faster before their switch (−70.8%). Accuracy for the phase-changing processes is 97%, overall 98.7%; the remaining errors fall almost entirely in the short reaction windows after each switch. The background batch tasks give up 15.9% of their CPU time.

6.4. ADVERSARIAL WORKLOAD AND ABLATION

This scenario adds two batch processes that imitate latency-sensitive behaviour, and runs a third mode, **noguard**, in which the anti-gaming guard is disabled (Table 6.4, Figure 6.4). Without the guard the classifier is fooled: only 0% of the adversaries' samples are classified correctly (Figure 6.5b), the adversaries receive the latency weight and finish 30.8% faster than in the baseline, and the genuine latency and interactive processes lose much of their benefit. With the guard, 100% of the adversaries' samples are classified as batch — none reach the latency class (Figure 6.5a). The genuine processes keep their full improvement (latency p95 −68.3% with the guard versus −30.2% without it), and the adversaries take 9.0% longer than in the baseline instead of gaining.

Table 6.4: Adversarial workload (mean ± 95% CI; Δ relative to baseline)

Metric	Baseline	Context-aware	No guard	Δ CA	Δ no guard
Latency wake-up p95 (ms)	9.17 ± 0.16	2.90 ± 0.53	6.40 ± 0.13	−68.3%	−30.2%
Latency wake-up p99 (ms)	11.24 ± 0.14	4.78 ± 0.77	8.76 ± 0.15	−57.5%	−22.1%
Interactive response p95 (ms)	105.51 ± 0.84	42.35 ± 1.18	67.59 ± 2.37	−59.9%	−35.9%
Adversary completion (s)	19.78 ± 0.11	21.55 ± 0.04	13.69 ± 0.13	+9.0%	−30.8%
Background batch CPU (ms)	3134 ± 30	2330 ± 23	2106 ± 34	−25.7%	−32.8%

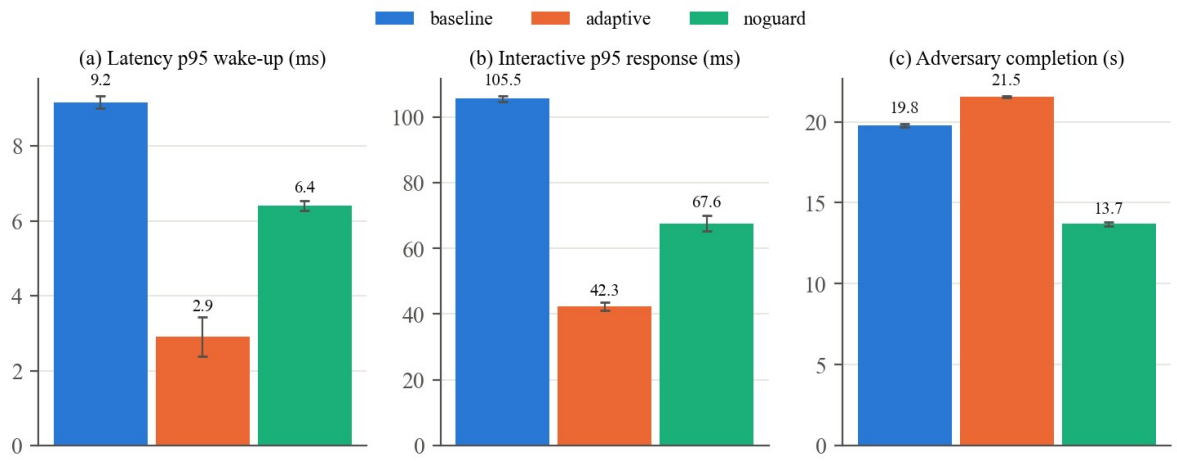


Figure 6.4: Adversarial workload: effect of the anti-gaming guard (error bars: 95% CI)

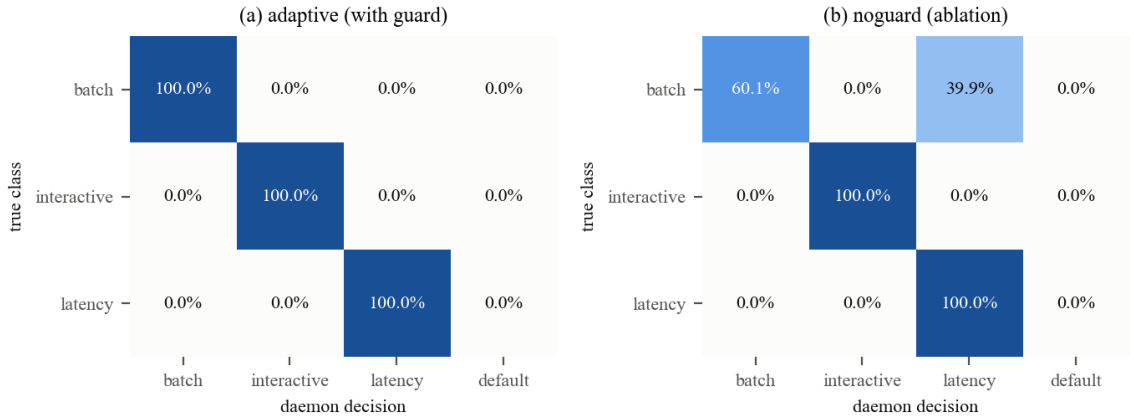


Figure 6.5: Confusion matrices of the classifier in the adversarial scenario, with and without the guard

6.5. REAL BENCHMARK PROGRAMS

The last scenario replaces the synthetic programs with unmodified benchmark tools (Table 6.5, Figure 6.6). The classifier labels them correctly (stress-ng 100%, fio 100%, sysbench 100%, cyclictest 100%, synthetic 100%). The storage benchmark **fio** benefits most: its synchronous writes wait for the CPU after every I/O completion, so placing it in the interactive class reduces its p99 write latency by 71.1% and raises its throughput by 15.8%. The synthetic interactive task again improves (−68.1%), while the CPU-bound tools pay a small price (stress-ng −3.9% operations per second, sysbench completion time +2.9%). **cyclictest** changes little (p99 +7.7%): it wakes 1000 times per second and runs for only microseconds, so EEVDF already serves it within about 126 µs at the default weight. The benefit of context-aware weights is therefore largest for tasks whose bursts are measured in milliseconds.

Table 6.5: Benchmark tools (mean ± 95% CI)

Metric	Baseline	Context-aware	Change
cyclictest wake-up p95 (ms)	0.126 ± 0.004	0.125 ± 0.006	−0.3%
cyclictest wake-up p99 (ms)	0.237 ± 0.027	0.256 ± 0.043	+7.7%
fio write latency p99 (ms)	0.789 ± 0.841	0.228 ± 0.017	−71.1%
fio IOPS (higher is better)	2174 ± 319	2519 ± 95	+15.8%
Interactive response p95 (ms)	61.05 ± 1.12	19.46 ± 0.29	−68.1%
stress-ng bogo ops/s (higher is better)	1548 ± 24	1488 ± 28	−3.9%
sysbench completion (s)	17.68 ± 0.19	18.20 ± 0.23	+2.9%

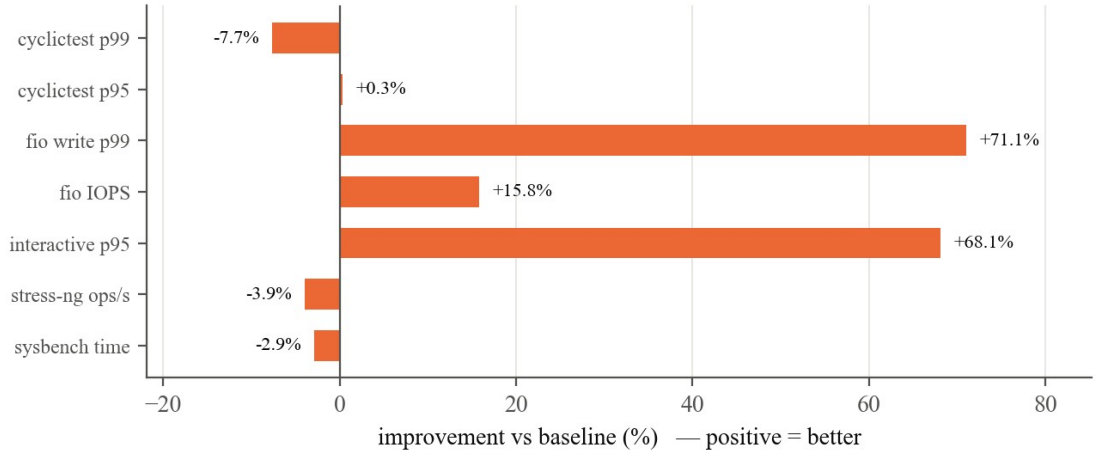


Figure 6.6: Benchmark tools: improvement of the context-aware system over the baseline

6.6. CLASSIFIER ACCURACY SUMMARY

Table 6.6: Accuracy of applied decisions on the real kernel (after a 1 s warm-up per process)

Scenario	Accuracy	Low-confidence fallback	Samples
mixed	100.0%	0.0%	20,611
phase	98.7%	0.1%	15,088
adversarial (with guard)	100.0%	0.0%	17,280
adversarial (no guard)	84.0%	0.0%	15,729
tools	100.0%	0.0%	13,116

Measured on the real kernel the classifier is more accurate than in the simulator (79.2%), because real interactive and latency-sensitive behaviour differ clearly in wake-up rate and burst length, while the simulator's parameter ranges overlapped. The low fallback rates show that the confidence threshold rarely needs to intervene for these workloads.

6.7. DISCUSSION

Trade-off. In every scenario the gains for interactive, latency-sensitive and I/O-bound processes are paid for by CPU-bound processes, by 16–26% of their CPU time or throughput when they compete with many favoured tasks on a single core (Tables 6.2–6.4), and by only 3–4% with the real benchmark tools (Table 6.5). This cost is predictable and controlled entirely by the weights in Table 4.1, which an administrator can tune.

Robustness. Three mechanisms made the classifier reliable in practice: the contention-independent *sleep_frac* signal (without it, latency tasks were misclassified under load), the I/O rule based on measured block-I/O wait (without it, fio looks latency-sensitive), and the guard (without it, a simple imitator wins the highest weight). Hysteresis costs about half a second of reaction time after a behaviour change. Because every threshold is tied to an external reference and every workload sits far from the thresholds (Table 4.5), the results do not depend on fine-tuning the numbers to our own workloads.

Overhead. The daemon runs outside the scheduling fast path and only reads a few small `/proc` files per process every 100 ms; it was pinned to a separate core in all experiments, and the context switches themselves are untouched.

Threats to validity. The experiments ran in a WSL2 virtual machine, so absolute latencies include hypervisor noise; comparisons remain valid because the modes alternate on the same machine. An adversarial run that overlapped with another experiment on the same core was detected (its baseline variance was abnormally high) and discarded; the adversarial scenario was repeated in isolation. Ground truth for the benchmark tools is a label chosen by us (for example fio counts as interactive because it is I/O-bound).

7. CONCLUSION

This project showed that a small user-space daemon can recognise the behaviour of running processes from kernel scheduler statistics alone and use cgroup v2 CPU weights to give each class appropriate treatment on an unmodified Linux kernel. On a real 6.6 kernel the 95th-percentile wake-up delay of latency-sensitive tasks fell by 76.2% and interactive response time by 52.4% under heavy contention, at a predictable cost of 19.5% in batch completion time. The classifier was 100% accurate on the mixed workload, followed behaviour changes within about 0.5 s, and — thanks to a guard based on a contention-independent sleep signal — could not be fooled by a batch job imitating a latency-sensitive one, whereas without the guard the imitator was never detected.

The work also exposed the limits of the approach: weights shift shares and deadlines but give no hard latency guarantee, tasks that are already served in microseconds (cyclictest) gain little, and classification is per process rather than per thread.

Future work

- Use per-task custom slices (Linux 6.12+) in addition to weights, to restore the simulator's per-class quantum directly.
- Classify individual threads using threaded cgroups, so that a process with both a latency thread and a batch thread can be served correctly.
- Learn the thresholds from data (for example a small decision tree trained on the logged signals) and compare it with the hand-written rules.
- Evaluate on bare-metal hardware and with real interactive applications such as a web server under load.

REFERENCES

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[2] F. J. Corbató, M. Merwin-Daggett, and R. C. Daley, “An experimental time-sharing system,” in *Proc. AFIPS Spring Joint Computer Conf.*, 1962, pp. 335–344.

[3] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, “Scheduling: The multi-level feedback queue,” in *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books, 2018.

[4] R. Love, *Linux Kernel Development*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2010.

[5] J. Roberson, “ULE: A modern scheduler for FreeBSD,” in *Proc. BSDCon*, San Mateo, CA, USA, 2003.

[6] The Linux Kernel Documentation, “CFS Scheduler.” [Online]. Available: https://docs.kernel.org/scheduler/sched-design-CFS.html

[7] I. Stoica and H. Abdel-Wahab, “Earliest eligible virtual deadline first: A flexible and accurate mechanism for proportional share resource allocation,” Old Dominion Univ., Norfolk, VA, USA, Tech. Rep. TR-95-22, 1995.

[8] J. Corbet, “An EEVDF CPU scheduler for Linux,” *LWN.net*, Mar. 2023. [Online]. Available: https://lwn.net/Articles/925371/

[9] Kernel Newbies, “Linux 6.12.” [Online]. Available: https://kernelnewbies.org/Linux_6.12

[10] The Linux Kernel Documentation, “Control Group v2.” [Online]. Available: https://docs.kernel.org/admin-guide/cgroup-v2.html

[11] The Linux Kernel Documentation, “Scheduler Statistics.” [Online]. Available: https://docs.kernel.org/scheduler/sched-stats.html

[12] M. Kerrisk, “proc(5) — Linux manual page.” [Online]. Available: https://man7.org/linux/man-pages/man5/proc.5.html

[13] R. Jain, D.-M. Chiu, and W. R. Hawe, “A quantitative measure of fairness and discrimination for resource allocation in shared computer systems,” Digital Equipment Corp., Tech. Rep. DEC-TR-301, 1984.

[14] The Linux Foundation, “rt-tests: cyclictest.” [Online]. Available: https://wiki.linuxfoundation.org/realtime/documentation/howto/tools/cyclictest/start

[15] C. I. King, “stress-ng.” [Online]. Available: https://github.com/ColinIanKing/stress-ng

[16] A. Kopytov, “sysbench.” [Online]. Available: https://github.com/akopytov/sysbench

[17] J. Axboe, “fio — Flexible I/O tester.” [Online]. Available: https://github.com/axboe/fio

[18] The FreeBSD Project, “sys/kern/sched_ule.c” (ULE scheduler source: sched_interact_score(), SCHED_INTERACT_THRESH). [Online]. Available: https://github.com/freebsd/freebsd-src/blob/main/sys/kern/sched_ule.c

[19] Linux kernel v6.6, “kernel/sched/fair.c” (sysctl_sched_base_slice, get_update_sysctl_factor()). [Online]. Available: https://github.com/torvalds/linux/blob/v6.6/kernel/sched/fair.c

[20] Linux kernel v6.6, “kernel/sched/core.c” (sched_prio_to_weight). [Online]. Available: https://github.com/torvalds/linux/blob/v6.6/kernel/sched/core.c

[21] The Linux Kernel Documentation, “Delay accounting.” [Online]. Available: https://docs.kernel.org/accounting/delay-accounting.html

[22] M. Kerrisk, “proc_pid_stat(5) — Linux manual page,” field (42) delayacct_blkio_ticks. [Online]. Available: https://man7.org/linux/man-pages/man5/proc_pid_stat.5.html

[23] R. B. Miller, “Response time in man-computer conversational transactions,” in *Proc. AFIPS Fall Joint Computer Conf.*, vol. 33, 1968, pp. 267–277.

[24] S. K. Card, G. G. Robertson, and J. D. Mackinlay, “The information visualizer, an information workspace,” in *Proc. ACM CHI Conf. Human Factors in Computing Systems*, 1991.

[25] J. Nielsen, *Usability Engineering*. San Diego, CA, USA: Academic Press, 1993, ch. 5. See also “Response times: The 3 important limits,” Nielsen Norman Group. [Online]. Available: https://www.nngroup.com/articles/response-times-3-important-limits/

[26] PipeWire, “pipewire.conf(5): default.clock.quantum, default.clock.rate.” [Online]. Available: https://docs.pipewire.org/page_man_pipewire_conf_5.html

[27] Y. Etsion, D. Tsafrir, and D. G. Feitelson, “Process prioritization using output production: Scheduling for multimedia,” *ACM Trans. Multimedia Computing, Communications and Applications*, vol. 2, pp. 318–342, 2006.

[28] H. Zheng and J. Nieh, “RSIO: Automatic user interaction detection and scheduling,” in *Proc. ACM SIGMETRICS*, 2010, pp. 263–274.

[29] D. C. Steere, A. Goel, J. Gruenberg, D. McNamee, C. Pu, and J. Walpole, “A feedback-driven proportion allocator for real-rate scheduling,” in *Proc. 3rd USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 1999, pp. 145–158.

[30] A. Negi and P. K. Kumar, “Applying machine learning techniques to improve Linux process scheduling,” in *Proc. IEEE TENCON*, 2005, doi: 10.1109/TENCON.2005.300837.

APPENDIX

A. How to run the system (inside Ubuntu on WSL2, or any Linux with cgroup v2)

```
cd /mnt/c/Users/<user>/vit/OS/project/linux
sudo ./setup.sh --install-tools      # cgroups, tools, build
sudo ./run_experiment.sh all 10 20   # 4 scenarios x 10 reps
python3 analyze.py results/<timestamp>

sudo ./ctxschedd --log /tmp/log.csv  # live mode
echo <PID> | sudo tee /sys/fs/cgroup/ctxsched/pending/cgroup.procs

gcc -O2 -o scheduler ../scheduler.c -lm && ./scheduler
```

B. The classifier (`classify()` in `linux/daemon.c`)

```
static Class classify(const Signals *s, double *conf) {
    double burst = s->burst_ms > 1e-6 ? s->burst_ms : 1e-6;
    double sleep = sleep_frac(s);
    double m_io  = have_delayacct
        ? fmin(lg2(s->io_wait / (s->cpu_share > 1e-6 ? s->cpu_share : 1e-6)),
               lg2(s->io_wait / IO_MIN_WAIT))
        : fmin((sleep - SLEEP_MIN) / 0.2, lg2(s->io_mbps / IO_MIN_MBPS));
    double m_lat = fmin(lg2(s->wake_s / LAT_MIN_WAKE), lg2(lat_max_burst / burst));
    if (guard) m_lat = fmin(m_lat, (sleep - SLEEP_MIN) / 0.2);
    double m_int = fmin((sleep - SLEEP_MIN) / 0.2, lg2(s->wake_s / INT_MIN_WAKE));

    Class c; double m;
    if (m_io > 0)       { c = C_INTERACTIVE; m = m_io; }
    else if (m_lat > 0) { c = C_LATENCY;     m = fmin(m_lat, -m_io); }
    else if (m_int > 0) { c = C_INTERACTIVE; m = fmin(fmin(m_int, -m_lat), -m_io); }
    else                { c = C_BATCH;       m = fmin(fmin(-m_lat, -m_int), -m_io); }
    *conf = m < 1.0 ? m : 1.0;
    return c;
}
```